

UNIVERSITAS NEGERI YOGYAKARTA
FAKULTAS TEKNIK
PROGRAM STUDI TEKNIK INDUSTRI

LAPORAN PRAKTIKUM
APLIKASI WEB DAN MOBILE

Semester Genap 2025/2026

**MODUL React Routing & Multi-Page App + Integrasi API –
Pertemuan 10**

Nama Mahasiswa : Syalfa Oktifa Nanda
NIM : 23051430023
Kelas : A
Tanggal Praktikum : 1 Mei 2026
Dosen Pengampu : Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT.

Dikumpulkan paling lambat 7 hari setelah pelaksanaan praktikum

A. IDENTITAS & INFORMASI MODUL

Mata Kuliah	Aplikasi Web dan Mobile
Kode Modul	Modul React Routing & Multi-Page App + Integrasi API (Pertemuan 10)
Topik Utama	React Routing & Multi-Page App + Integrasi API
Sub-Topik	Single Page Application (SPA) Navigation, React Router DOM, dan Fetch API dalam React.
Durasi Praktikum	340 Menit
Tanggal Praktikum	1 Mei 2026
Nama Mahasiswa	Syalfa Oktifa Nanda
NIM	23051430023
Kelas	A

B. TUJUAN PEMBELAJARAN

1	Memahami konsep SPA (Single Page Application).
2	Mampu mengatur navigasi antar halaman tanpa reload browser menggunakan `react-router-dom`.
3	Menerapkan `fetch` API di dalam `useEffect` untuk mengambil data eksternal.
4	Membuat struktur aplikasi multi-halaman (Dashboard, Inventori, Settings).

C. ALAT DAN BAHAN

Sebutkan seluruh perangkat keras, perangkat lunak, dan sumber referensi yang digunakan selama praktikum.

No.	Nama Alat / Bahan / Aplikasi	Versi / Spesifikasi	Keterangan
1	Visual Studio Code (VS Code)	Versi 1.8x ke atas	Code Editor
2	Google Chrome / Browser Modern	Versi terbaru	Testing & Debugging
3	Node.js & npm	Node 18+ / npm 9+	Runtime JS (Modul tertentu)
4	React Router DOM	Versi 6+	Routing SPA

5	JSONPlaceholder API	Api Online	Simulasi data inventori
---	---------------------	------------	-------------------------

D. DASAR TEORI

D.1 Konsep Utama

React Router merupakan library pada React yang digunakan untuk mengatur navigasi antar halaman pada aplikasi berbasis Single Page Application (SPA). Dengan konsep SPA, perpindahan halaman dapat dilakukan tanpa melakukan reload browser sehingga aplikasi terasa lebih cepat dan responsif. Navigasi ini dilakukan menggunakan react-router-dom melalui komponen seperti BrowserRouter, Routes, Route, dan Link.

Pada praktikum ini juga digunakan fetch API untuk mengambil data dari sumber eksternal. Fetch API bekerja secara asynchronous sehingga data dapat dimuat tanpa menghentikan proses utama aplikasi. Penggunaan useEffect memungkinkan proses pengambilan data dijalankan ketika komponen pertama kali dirender. Data yang berhasil diambil kemudian disimpan ke dalam state menggunakan useState.

Selain itu, aplikasi dibangun dengan struktur multi-page seperti Dashboard, Inventori, dan halaman Detail Inventori. Struktur modular ini membuat kode lebih rapi, mudah dikembangkan, dan mempermudah proses maintenance. Adanya halaman NotFound juga membantu menangani URL yang tidak tersedia sehingga pengalaman pengguna menjadi lebih baik.

D.2 Keterkaitan dengan Teknik Industri

Materi React Routing dan integrasi API sangat relevan dengan bidang Teknik Industri, khususnya pada pengembangan sistem informasi industri. Dalam dunia industri, perusahaan membutuhkan aplikasi yang mampu menampilkan data produksi, inventori, kualitas, dan monitoring secara cepat tanpa reload halaman agar proses kerja lebih efisien.

Contoh penerapannya adalah sistem monitoring gudang dan inventori bahan baku pada pabrik. Data stok dapat diambil langsung dari database menggunakan API lalu ditampilkan secara realtime pada dashboard. Selain itu, navigasi antar halaman seperti dashboard produksi, laporan kualitas, dan data supplier dapat dilakukan dengan cepat menggunakan konsep SPA.

Teknologi ini juga dapat diterapkan pada sistem quality control di industri manufaktur. Data cacat produk dapat ditampilkan dalam bentuk laporan digital sehingga mempermudah analisis dan pengambilan keputusan. Dengan sistem berbasis web modern, proses monitoring menjadi lebih efektif dan mendukung transformasi industri menuju digitalisasi.

E. LANGKAH KERJA DAN HASIL PRAKTIKUM

E.1 Langkah 1 – Setup Router

Deskripsi singkat apa yang dilakukan pada langkah ini:

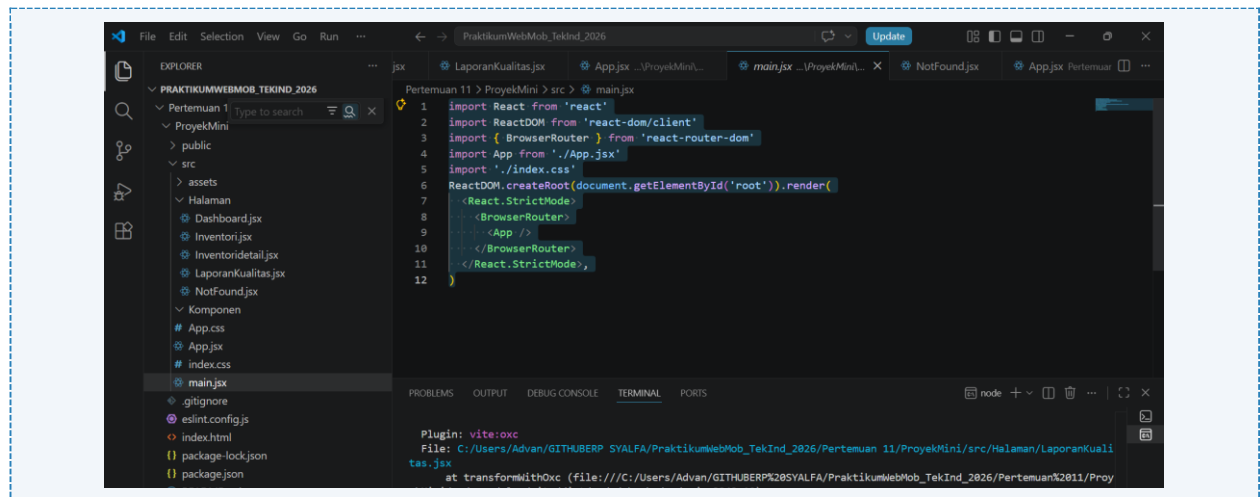
Pada langkah ini dilakukan konfigurasi awal React Router dengan membungkus komponen utama aplikasi menggunakan BrowserRouter pada file main.jsx. Tujuannya agar aplikasi dapat mendukung navigasi antar halaman berbasis SPA tanpa reload browser.

```

/* ===== KODE / OUTPUT LANGKAH 1 ===== */
import React from 'react'
import ReactDOM from 'react-dom/client'
import { BrowserRouter } from 'react-router-dom'
import App from './App.jsx'
import './index.css'
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>,
)

```

Screenshot Hasil:



Gambar E.1 – Konfigurasi BrowserRouter Pada Main.jsx

E.2 Langkah 2 – Membuat Halaman (Pages)

Pada langkah ini dibuat beberapa halaman utama aplikasi yaitu Dashboard.jsx, Inventori.jsx, dan NotFound.jsx. Halaman Dashboard digunakan sebagai halaman utama sistem, halaman Inventori digunakan untuk menampilkan data inventori dari API eksternal menggunakan fetch API, sedangkan halaman NotFound digunakan untuk menampilkan error 404 ketika URL tidak ditemukan.

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
src/Halaman/Dashboard.jsx
import React from 'react';
import { Link } from 'react-router-dom';
function Dashboard() {
  return (
    <div className="p-5">
      <h1>Dashboard Utama Pabrik</h1>
      <p>Selamat datang di sistem monitoring terpadu.</p>
    </div>
  );
}

```

```
        <Link to="/inventori" className="btn btn-primary">
            Ke Halaman Inventori
        </Link>
    </div>
    );
}
export default Dashboard;
```

```
/* ===== KODE / OUTPUT LANGKAH 2 ===== */
src/Halaman/Inventori.jsx
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';
function Inventori() {
    const [products, setProducts] = useState([]);
    // Fetch Data saat komponen mount
    useEffect(() => {
        fetch('https://jsonplaceholder.typicode.com/posts')
            .then(res => res.json())
            .then(data => {
                // Ambil hanya 5 data pertama untuk contoh
                setProducts(data.slice(0, 5));
            })
            .catch(err => console.log(err));
    }, []);
    return (
        <div className="container mt-4">
            <h1>Data Inventori Bahan Baku</h1>
            <Link to="/" className="btn btn-secondary mb-3">Kembali ke
Dashboard</Link>
            <table className="table table-striped">
                <thead>
                    <tr>
                        <th>ID Item</th>
                        <th>Nama Bahan</th>
                        <th>Status Supplier</th>
                    </tr>
                </thead>
                <tbody>
                    {products.map((item) => (
                        <tr key={item.id}>
                            <td>{item.id}</td>
                            <td>{item.title}</td> { /* Menggunakan title
sebagai simulasi nama bahan */}
```

```

                                <td><span className="badge bg-
success">Available</span></td>
                                </tr>
                                )})
                                </tbody>
                                </table>
                                </div>
                                );
                                }
                                export default Inventori;

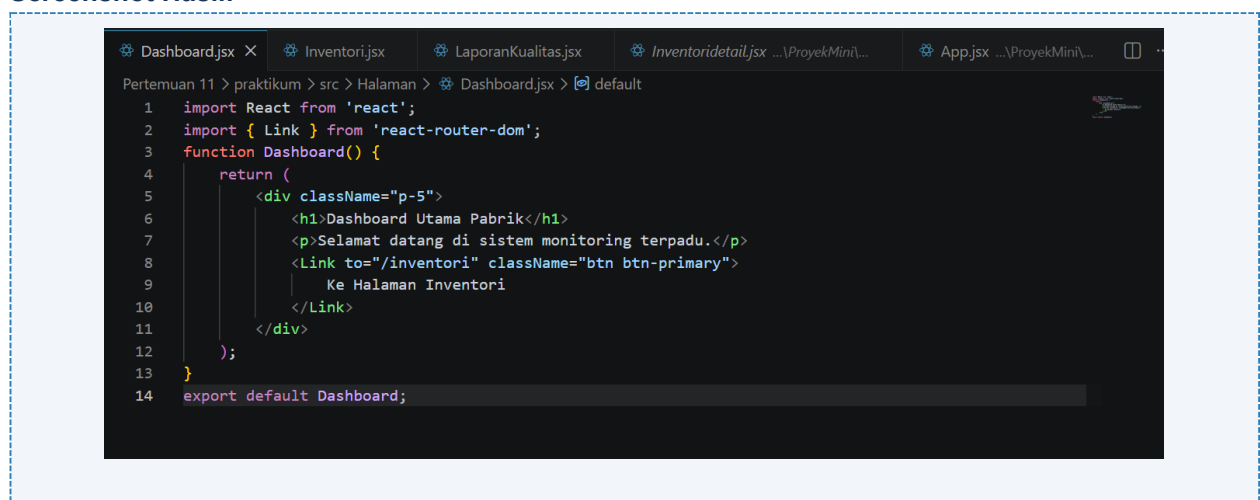
```

```

/* ===== KODE / OUTPUT LANGKAH 2 ===== */
src/Halaman/NotFound.jsx
import React from 'react';
import { Link } from 'react-router-dom';
function NotFound() {
    return (
        <div className="text-center mt-5">
            <h1 className="display-1">404</h1>
            <p>Halaman tidak ditemukan dalam sistem pabrik.</p>
            <Link to="/" className="btn btn-dark">Kembali ke Home</Link>
        </div>
    );
}
export default NotFound;

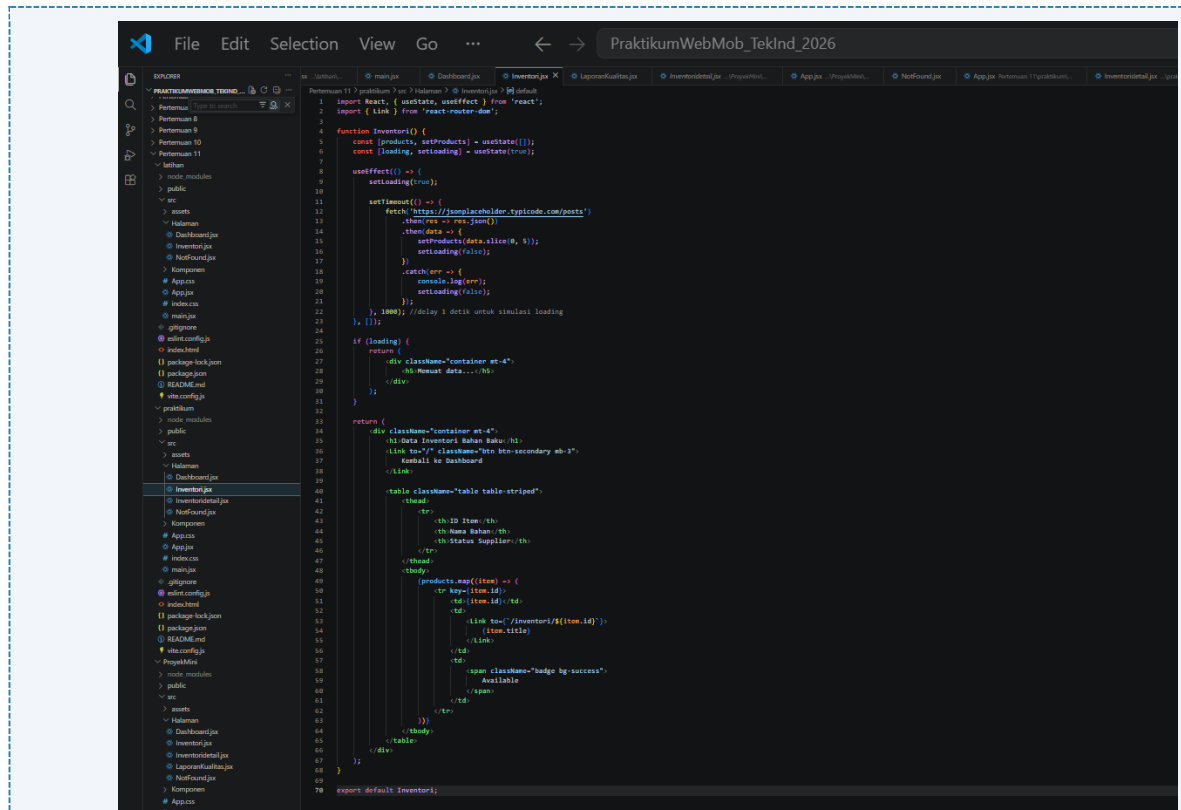
```

Screenshot Hasil:

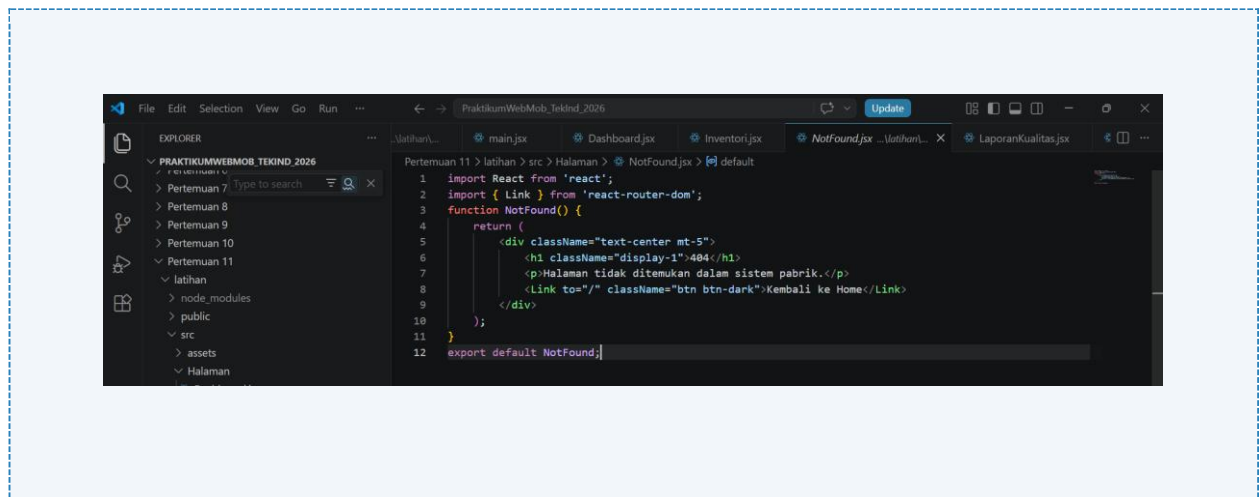


Gambar E.2 – Tampilan Codingan Halaman Dashborad

Screenshot Hasil:



Screenshot Hasil:

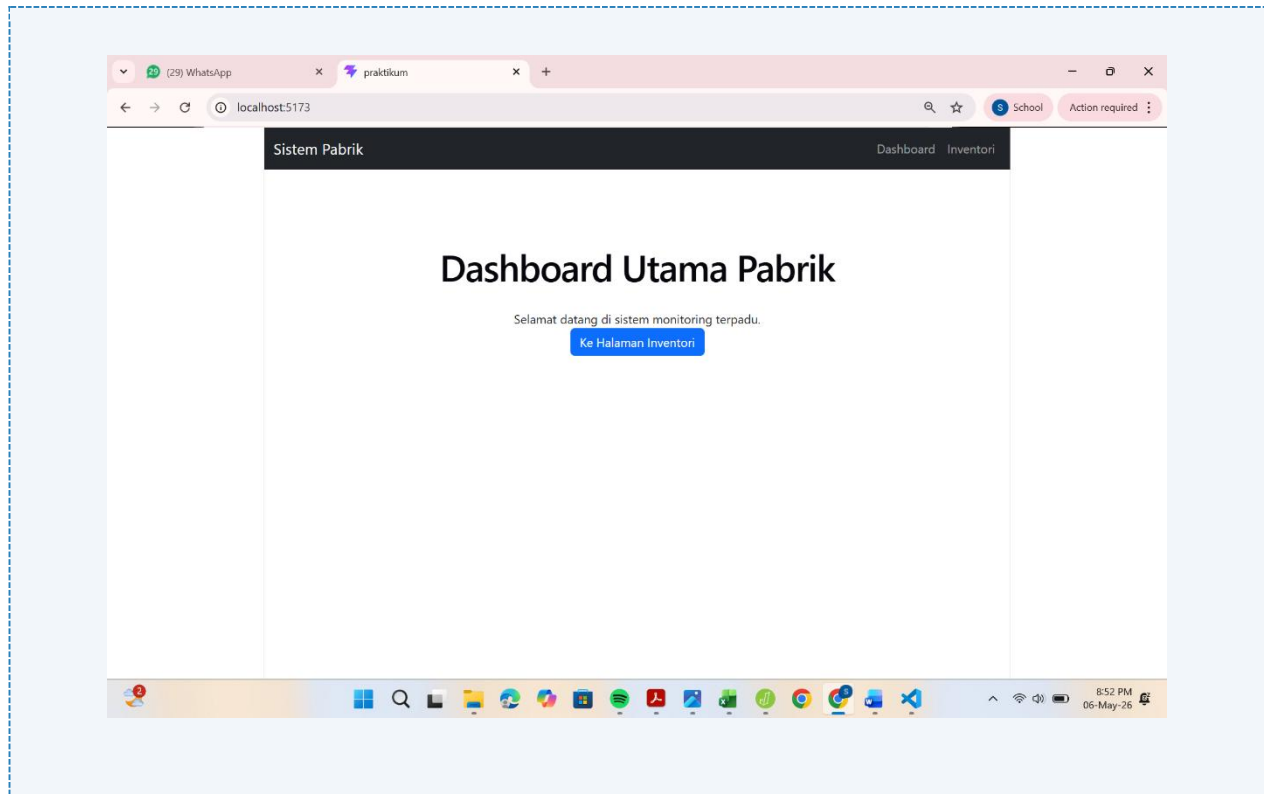


E.3 Langkah 3 – Mengatur Route di App.jsx

Pada langkah ini dilakukan pengaturan route menggunakan komponen Routes dan Route dari react-router-dom. Selain itu dibuat navbar sederhana untuk mempermudah navigasi antar halaman seperti Dashboard dan Inventori. Route tambahan juga dibuat untuk halaman detail inventori dan halaman error 404.

```
/* ===== KODE / OUTPUT LANGKAH 3 ===== */
import React from 'react';
import { Routes, Route } from 'react-router-dom';
import Dashboard from './Halaman/Dashboard';
import Inventori from './Halaman/Inventori';
import NotFound from './Halaman/NotFound';
// import Navbar from './Komponen/Navbar'; // Anda bisa buat navbar sendiri
import Inventoridetail from './Halaman/Inventoridetail';
// Buat Navbar.jsx sederhana untuk navigasi
function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark mb-4">
      <div className="container">
        <a className="navbar-brand" href="/">Sistem Pabrik</a>
        <div className="navbar-nav">
          <a className="nav-link" href="/">Dashboard</a>
          <a className="nav-link" href="/inventori">Inventori</a>
        </div>
      </div>
    </nav>
  )
}
function App() {
  return (
    <div>
      <Navbar />
      <Routes>
        {/* Route yang tepat akan di-render */}
        <Route path="/" element={<Dashboard />} />
        <Route path="/Inventori" element={<Inventori />} />
        <Route path="/Inventori/:id" element={<Inventoridetail />} />
        {/* Route untuk semua path lainnya (404) */}
        <Route path="*" element={<NotFound />} />
      </Routes>
    </div>
  );
}
export default App;
```

Screenshot Hasil:



Gambar E.3 – Tampilan Halaman dashboard inventori

F. LATIHAN DAN TUGAS MANDIRI

F.1 Latihan 1 – Dynamic Route

Pertanyaan / Instruksi Latihan:

Latihan 1 (Dynamic Route): Tambahkan Route untuk detail item. Misal path `/inventori/:id`. Di halaman Inventori, ubah "Nama Bahan" menjadi Link ke halaman detail.

Jawaban / Kode Solusi:

```
import React, { useEffect, useState } from 'react';
import { useParams, Link } from 'react-router-dom';

function InventoriDetail() {
  const { id } = useParams();
  const [item, setItem] = useState(null);

  useEffect(() => {
    fetch(`https://jsonplaceholder.typicode.com/posts/${id}`)
      .then(res => res.json())
      .then(data => setItem(data))
      .catch(err => console.log(err));
  }, [id]);
```

```

if (!item) return <p>Loading...</p>;

return (
  <div className="container mt-5">
    <div className="row justify-content-center">
      <div className="col-lg-7">

        <div className="card border-0 shadow-lg" style={{
borderRadius: '16px' }}>

          { /* Header */ }
          <div
            className="p-4 text-white"
            style={{
              background: 'linear-gradient(135deg, #1e3c72,
#2a5298)',
              borderTopLeftRadius: '16px',
              borderTopRightRadius: '16px'
            }}
          >
            <h4 className="mb-0 fw-bold">Detail Bahan</h4>
            <small className="opacity-75">Informasi lengkap
material</small>
          </div>

          { /* Body */ }
          <div className="p-4">

            <div className="d-flex justify-content-between
mb-3">

              <div>
                <small className="text-muted">ID</small>
                <div className="fw-bold">{item.id}</div>
              </div>

              <span className="badge bg-success align-self-
start">
                Available
              </span>
            </div>

            <hr />

            <div className="mb-3">
              <small className="text-muted">Nama
Bahan</small>

```

```
        <div className="fs-5 fw-semibold">
            {item.title}
        </div>
    </div>

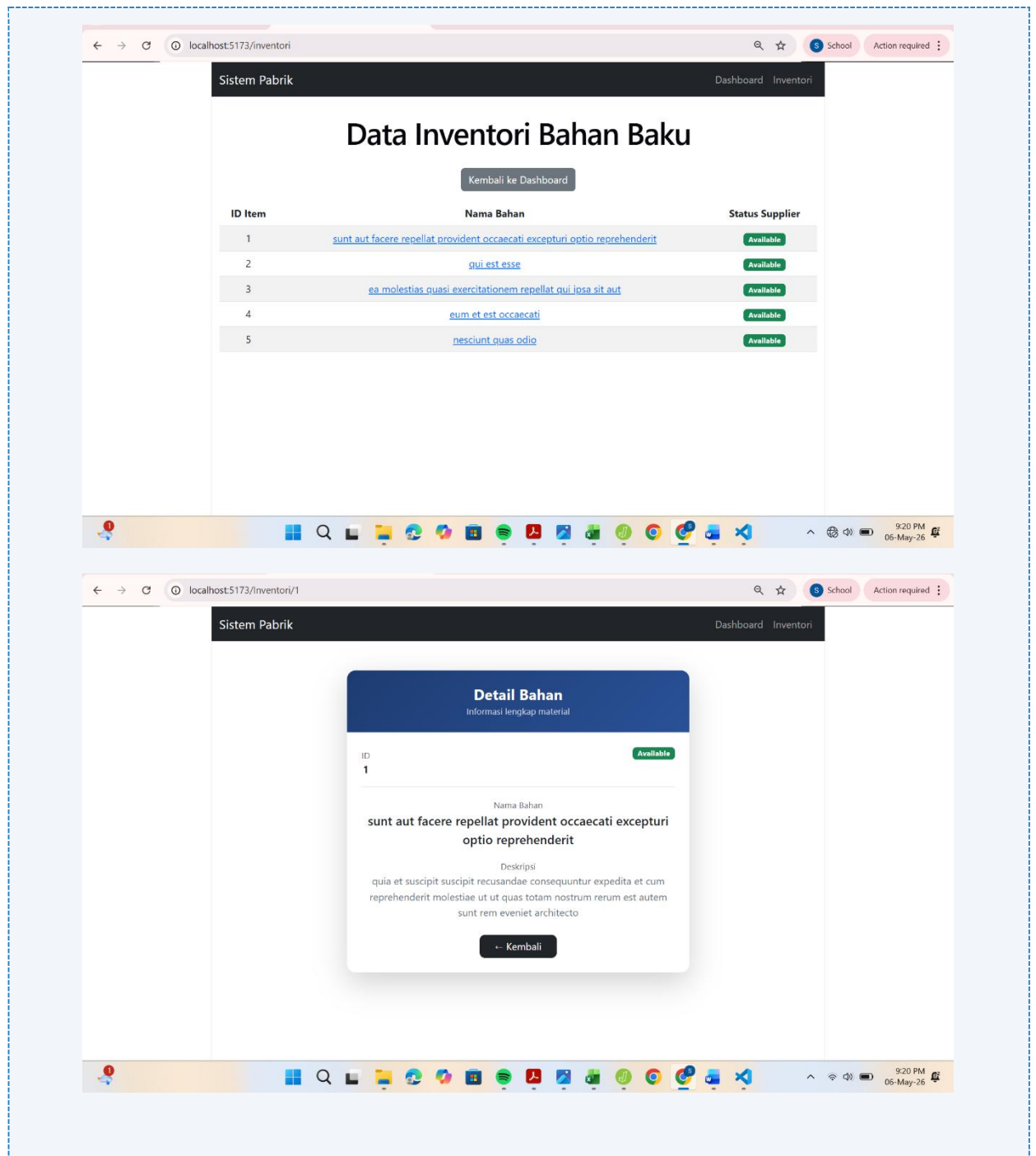
    <div className="mb-4">
        <small className="text-
muted">Deskripsi</small>
        <p className="mb-0 text-secondary" style={{
lineHeight: '1.6' }}>
            {item.body}
        </p>
    </div>

    { /* Action */ }
    <Link
        to="/inventori"
        className="btn btn-dark px-4"
        style={{ borderRadius: '10px' }}
    >
        ← Kembali
    </Link>

    </div>
</div>

</div>
</div>
</div>
);
}

export default InventoriDetail;
```



Gambar F.1 – Tampilanm Halaman Detail

F.2 Latihan 2 – Loading State

Pertanyaan / Instruksi Latihan:

Latihan 2 (Loading State): Di halaman `Inventori.jsx`, tambahkan state `loading` (boolean). Set `loading = true` sebelum fetch, dan `false` setelah data masuk. Tampilkan teks "Memuat data..." jika loading true.

Jawaban / Kode Solusi:

```
import React, { useState, useEffect } from 'react';
import { Link } from 'react-router-dom';

function Inventori() {
  const [products, setProducts] = useState([]);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    setLoading(true);

    setTimeout(() => {
      fetch('https://jsonplaceholder.typicode.com/posts')
        .then(res => res.json())
        .then(data => {
          setProducts(data.slice(0, 5));
          setLoading(false);
        })
        .catch(err => {
          console.log(err);
          setLoading(false);
        });
    }, 1000); //delay 1 detik untuk simulasi loading
  }, []);

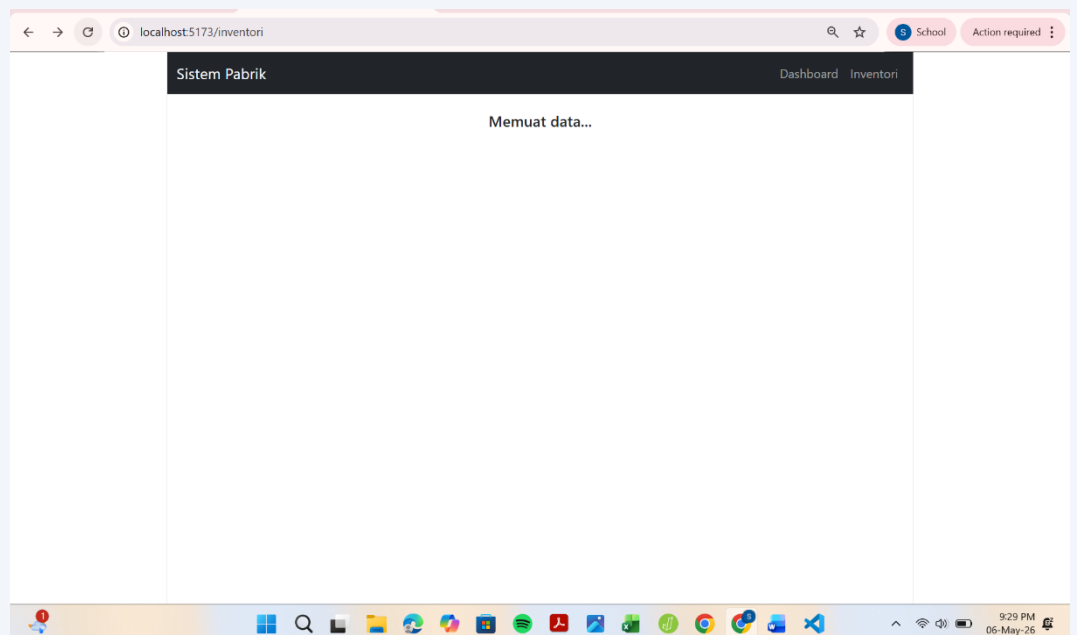
  if (loading) {
    return (
      <div className="container mt-4">
        <h5>Memuat data...</h5>
      </div>
    );
  }

  return (
    <div className="container mt-4">
      <h1>Data Inventori Bahan Baku</h1>
      <Link to="/" className="btn btn-secondary mb-3">
        Kembali ke Dashboard
      </Link>

      <table className="table table-striped">
        <thead>
          <tr>
            <th>ID Item</th>
            <th>Nama Bahan</th>
            <th>Status Supplier</th>
          </tr>
        </thead>
      </table>
    </div>
  );
}
```

```
    </thead>
    <tbody>
      {products.map((item) => (
        <tr key={item.id}>
          <td>{item.id}</td>
          <td>
            <Link to={`/inventori/${item.id}`}>
              {item.title}
            </Link>
          </td>
          <td>
            <span className="badge bg-success">
              Available
            </span>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
</div>
);
}

export default Inventori;
```



Gambar F.2 – Tampilan Loading State

F.3 Tugas Proyek Mini – Navigasi Modular

Deskripsi proyek mini:

Tugas Proyek Mini (Navigasi Modular):

- Buat halaman baru bernama `LaporanKualitas.jsx`. Halaman ini menampilkan data "Cacat" (mock data array manual).
- Tambahkan link di Navbar untuk menuju halaman ini.
- Pastikan transisi antar halaman berjalan mulus tanpa reload browser

Penjelasan pendekatan/solusi yang digunakan:

Pada proyek mini ini dibuat halaman baru bernama LaporanKualitas.jsx yang digunakan untuk menampilkan data cacat produk menggunakan array manual (mock data). Data ditampilkan dalam bentuk tabel agar lebih mudah dibaca dan dianalisis.

Selanjutnya ditambahkan menu navigasi baru pada Navbar menggunakan Link dari react-router-dom sehingga pengguna dapat berpindah halaman tanpa reload browser. Dengan pendekatan SPA, transisi halaman menjadi lebih cepat dan pengalaman pengguna menjadi lebih baik.

Selain itu ditambahkan badge status berdasarkan jumlah cacat produk. Jika jumlah cacat lebih dari 5 maka status ditampilkan sebagai "Tinggi", sedangkan jika kurang dari atau sama dengan 5 maka status ditampilkan sebagai "Normal". Hal ini membantu visualisasi kondisi kualitas produk secara lebih jelas.

Kode Utama:

```
import React from 'react';

function LaporanKualitas() {
  const dataCacat = [
    { id: 1, produk: 'Filter Oli', jenis: 'Korosi', jumlah: 6 },
    { id: 2, produk: 'Pompa Hidrolik', jenis: 'Deformasi', jumlah: 2 },
    { id: 3, produk: 'Bantalan Roda', jenis: 'Porositas', jumlah: 11 },
  ];

  return (
    <div className="container mt-5">
      <div className="row justify-content-center">
        <div className="col-lg-9">

          <div className="card border-0 shadow-lg" style={{
borderRadius: '16px' }}>

            {/* Header */}
            <div
              className="p-4 text-white"
              style={{
                background: 'linear-gradient(135deg, #1e3c72,
#2a5298)',
```

```

        borderTopLeftRadius: '16px',
        borderTopRightRadius: '16px'
      }}
    >
    <h4 className="mb-1 fw-bold">Laporan
Kualitas</h4>
    <small className="opacity-75">Data cacat produk
produksi</small>
  </div>

  { /* Body */ }
  <div className="p-4">

    <table className="table align-middle ">
      <thead className="table-info">
        <tr>
          <th>ID</th>
          <th>Produk</th>
          <th>Jenis Cacat</th>
          <th className="text-
center">Jumlah</th>
          <th className="text-
center">Status</th>
        </tr>
      </thead>

      <tbody>
        {dataCacat.map(item => (
          <tr key={item.id}>
            <td>{item.id}</td>
            <td className="fw-
semibold">{item.produk}</td>
            <td>{item.jenis}</td>
            <td className="text-center fw-
bold">
              {item.jumlah}
            </td>
            <td className="text-center">
              <span className={`badge
${item.jumlah > 5 ? 'bg-danger' : 'bg-success'}
              `}>
                {item.jumlah > 5 ?
'Tinggi' : 'Normal'}
              </span>

```

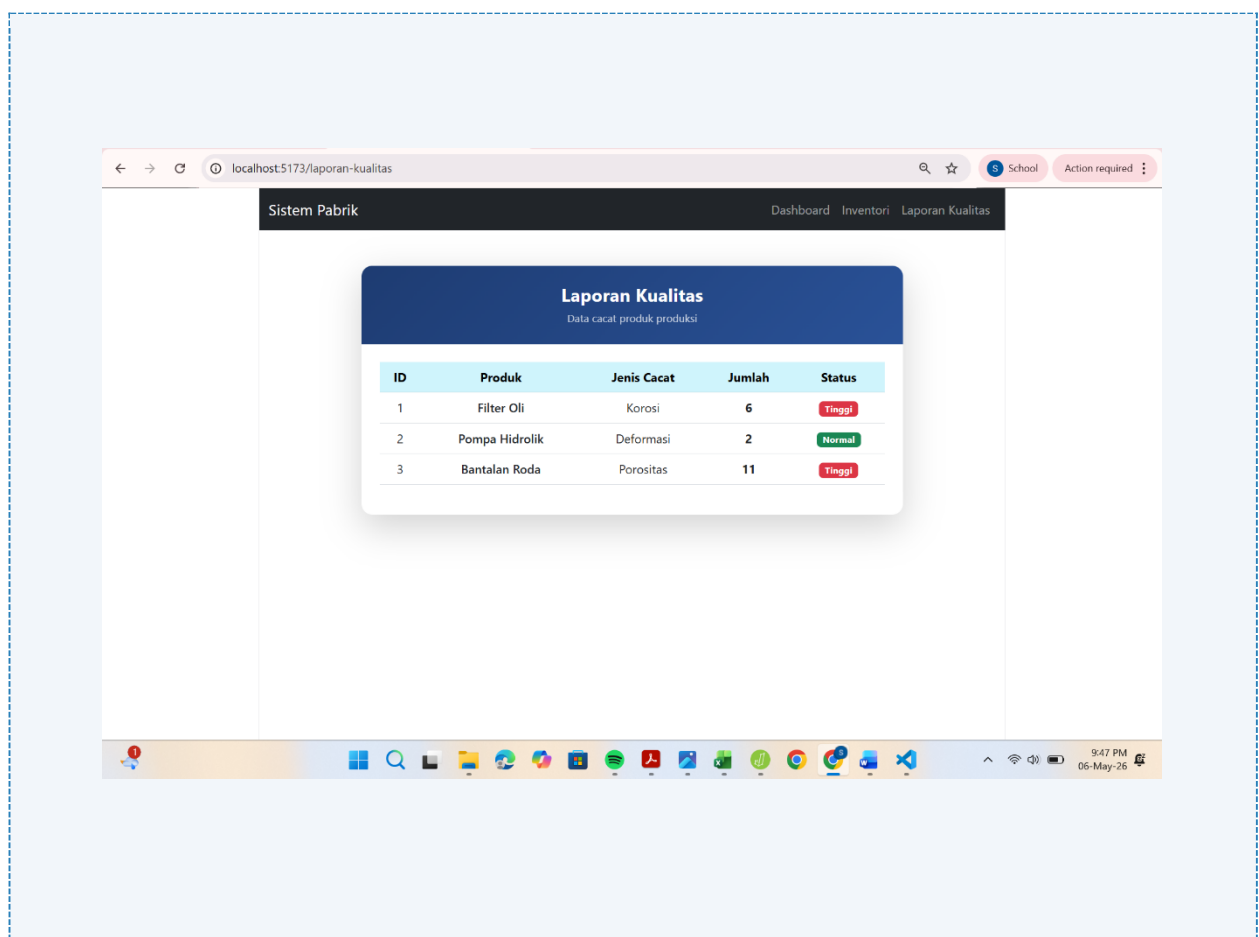


```
        </td>
      </tr>
    )})
  </tbody>
</table>

</div>
</div>

</div>
</div>
</div>
);
}

export default LaporanKualitas;
```



Gambar F.3 – Tampilan Halama Laporan Kualitas

G. PEMBAHASAN

G.1 Analisis Kesesuaian Hasil dengan Teori

[Hasil praktikum yang diperoleh telah sesuai dengan teori mengenai Single Page Application (SPA) dan React Routing. Perpindahan antar halaman Dashboard, Inventori, dan Laporan Kualitas dapat dilakukan tanpa reload browser karena menggunakan react-router-dom. Hal ini membuktikan bahwa konsep SPA berhasil diterapkan pada aplikasi.

Penggunaan fetch API pada halaman Inventori juga berjalan dengan baik. Data berhasil diambil dari API eksternal dan ditampilkan ke dalam tabel menggunakan state React. Implementasi useEffect dengan dependency array kosong membuat proses fetch hanya berjalan sekali saat komponen pertama kali dimuat.

Selain itu, penerapan dynamic route pada halaman detail inventori berhasil menampilkan detail data berdasarkan parameter id pada URL. Halaman loading state juga berfungsi dengan baik sehingga pengguna mendapatkan informasi saat data sedang dimuat.

G.2 Kendala yang Ditemukan dan Solusinya

Kendala / Error yang Ditemukan	Solusi yang Diterapkan
Navbar tidak tampil di semua halaman	Meletakkan komponen Navbar di luar komponen Routes
Loading terlalu cepat sehingga tidak terlihat	Menambahkan setTimeout untuk simulasi loading
Halaman tidak berpindah tanpa reload	Mengganti tag <a> menjadi komponen <Link> dari react-router-dom

G.3 Kaitan dengan Penerapan di Industri

[Materi pada praktikum ini dapat diterapkan pada pengembangan sistem informasi industri seperti dashboard monitoring produksi, sistem inventori gudang, dan laporan kualitas produk. Dengan React Router, perusahaan dapat membuat aplikasi web yang cepat dan responsif untuk mempermudah monitoring operasional.

Contohnya pada industri manufaktur, operator dapat melihat data inventori bahan baku secara realtime tanpa perlu reload halaman. Sistem quality control juga dapat memanfaatkan halaman laporan kualitas untuk memantau jumlah produk cacat dan mengambil tindakan perbaikan dengan lebih cepat.

Selain itu, penggunaan API memungkinkan aplikasi terhubung dengan database atau server perusahaan sehingga data dapat diperbarui secara otomatis. Hal ini mendukung digitalisasi industri dan meningkatkan efisiensi kerja pada era industri modern

H. KESIMPULAN

1	Praktikum ini berhasil menerapkan konsep Single Page Application menggunakan React dan react-router-dom.
2	Navigasi antar halaman dapat dilakukan tanpa reload browser sehingga aplikasi menjadi lebih cepat dan responsif.
3	Fetch API berhasil digunakan untuk mengambil data eksternal dan menampilkannya pada halaman inventori.
4	Dynamic route dan loading state berhasil diterapkan untuk meningkatkan interaktivitas aplikasi.
5	Materi React Routing sangat bermanfaat dalam pengembangan sistem informasi industri seperti dashboard monitoring dan manajemen inventori.

I. DAFTAR PUSTAKA

No.	Referensi (Format APA 7th Edition)
[1]	Burhandenny, A. E. (2026). Manual Praktikum Aplikasi Web dan Mobile Semester Genap 2025/2026. Program Studi Teknik Industri, Universitas Negeri Yogyakarta.
[2]	React Documentation. (2026). <i>React Router Documentation</i> . React.dev. https://reactrouter.com

J. LEMBAR PENILAIAN DAN PENGESAHAN

RUBRIK PENILAIAN LAPORAN					
Aspek Penilaian		Bobot			
No.	Aspek Penilaian	Bobot (%)	Nilai (0-100)	Skor Akhir	Catatan Penilai
1	Kelengkapan Isi (Semua bagian A–I terisi lengkap)	20%			
2	Kualitas Dasar Teori (Ditulis dengan bahasa sendiri, relevan, minimal 3 paragraf)	15%			
3	Dokumentasi Langkah Kerja (Kode & Screenshot sesuai, jelas, dan terbaca)	25%			
4	Tugas/Latihan Mandiri (Semua latihan & proyek mini dikerjakan dan berfungsi)	20%			
5	Pembahasan & Analisis (Kritis, mendalam, dan relevan dengan industri)	15%			
6	Kesimpulan & Tata Bahasa (Spesifik, rapi, mengikuti format yang ditentukan)	5%			
TOTAL NILAI AKHIR		100%			

Mahasiswa	Diperiksa oleh Asisten / Dosen
 Syalfa Oktifa Nanda NIM: 23051430023	Dr. Eng. Ir. Aji Ery Burhandenny, S.T., M.AIT. Tanggal: _____

PANDUAN DAN KETENTUAN PENGISIAN LAPORAN

Baca dengan seksama sebelum mengisi template laporan ini

1. Aturan Umum

a.	Laporan dikerjakan secara INDIVIDUAL, kecuali ada instruksi khusus dari dosen.
b.	Isi laporan harus merupakan hasil pekerjaan sendiri. Plagiarisme (menyalin laporan teman secara identik) akan mendapatkan nilai NOL (0) untuk kedua pihak.
c.	Bagian Dasar Teori HARUS ditulis dengan kalimat dan bahasa sendiri, bukan disalin mentah dari modul atau internet.
d.	Screenshot WAJIB memperlihatkan nama/NIM mahasiswa pada salah satu bagian tampilan (bisa di judul halaman web atau bagian lain) sebagai bukti keaslian pekerjaan.
e.	Semua teks dalam kotak yang berwarna abu-abu/biru muda dan bersifat miring adalah instruksi yang HARUS diganti dengan isi yang sesungguhnya.

2. Format File dan Ketentuan Pengumpulan

Hal	Ketentuan
Format File	Microsoft Word (.docx). Konversi ke PDF jika diminta dosen.
Nama File	Laporan_Modul[No]_[NIM]_[NamaLengkap].docx Contoh: Laporan_Modul02_12320001_AhmadDhani.docx
Ukuran Kertas	A4 (210 × 297 mm)
Margin	Atas: 3 cm Bawah: 3 cm Kiri: 4 cm Kanan: 3 cm
Font Teks Isi	Arial, ukuran 11pt, spasi 1.15
Font Kode Program	Courier New, ukuran 10pt, latar gelap (sudah tersedia di template)
Batas Pengumpulan	7 hari kalender setelah tanggal pelaksanaan praktikum
Media Pengumpulan	Github repository mahasiswa + konfirmasi email ke dosen
Subjek Email	[WebLab-No.Modul] NIM - Nama Lengkap Contoh: [WebLab-02] 12320001 - Ahmad Dhani

3. Kompilasi Laporan Akhir Semester

PENTING – LAPORAN AKHIR SEMESTER

Pada akhir semester, seluruh laporan modul (Modul 1–12, kecuali Modul 13 UTS dan Modul 16 UAS) akan dikompilasi menjadi SATU BERKAS LAPORAN AKHIR. Ketentuan kompilasi:

a.	Laporan setiap modul dicetak/disimpan sebagai satu dokumen terpisah sesuai template ini.
b.	Kompilasi disatukan dengan urutan: Halaman Cover Akhir → Daftar Isi → Laporan Modul 1 → Laporan Modul 2 → ... → Laporan Modul 12 → Laporan Modul 14 & 15.

c.	Laporan UTS (Modul 13) dan UAS (Modul 16) dikumpulkan secara TERPISAH dengan ketentuan yang diumumkan saat ujian.
d.	Setiap laporan modul di dalam kompilasi diawali dengan halaman Cover (Bagian A di template ini) sebagai pemisah antar modul.
e.	Nama file laporan akhir: LaporanAkhir_[NIM]_[NamaLengkap].docx
f.	Laporan akhir dikumpulkan paling lambat H-3 sebelum UAS.

4. Ketentuan Pengumpulan via GitHub

```

PraktikumWebMob_TekInd_2026/
├── README.md
├── Laporan/
│   ├── Laporan_Modul01_[NIM].docx
│   ├── Laporan_Modul02_[NIM].docx
│   └── ... (dst hingga Modul 15)
├── Pertemuan01_[NIM]_[Nama]/
│   ├── index.html
│   └── style.css
├── Pertemuan02_[NIM]_[Nama]/
└── ... (dst)

```

Pastikan setiap Commit memiliki pesan yang deskriptif. Contoh: "Menambahkan Laporan Modul 02 – CSS Dasar". Lakukan Push ke GitHub setiap akhir praktikum.

5. Konsekuensi Keterlambatan Pengumpulan

Keterlambatan	Konsekuensi
1–2 hari	Pengurangan nilai 10% dari total skor laporan
3–4 hari	Pengurangan nilai 25% dari total skor laporan
5–7 hari	Pengurangan nilai 50% dari total skor laporan
> 7 hari	Laporan TIDAK DITERIMA (nilai 0) – wajib menemui dosen untuk klarifikasi
Tidak dikumpulkan	Nilai laporan = 0, mahasiswa tidak diperkenankan mengikuti UAS

— Selamat mengerjakan —